



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251193
Nama Lengkap	Cheila Zefanya Wibowo
Minggu ke / Materi	13/Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Tuple Immutable

Tuple bisa dibilang lumayan mirip dengan list karena dapat menyimpan berbagai jenis nilai dan memiliki indeks berupa bilangan bulat (integer). Perbedaan nya adalah sifat tuple yang immutable yaitu dimana anggota dalam tuple tidak dapat dirubah setelah dibuat. Tuple dapat dibandingkan dan bersifat hashable, sehingga dapat digunakan sebagai kunci dalam dictionary python.

Hashable itu objek yang memiliki nilai hash yang tidak berubah selama masa hidupnya (menggunakan metode `hash_()`), dan dapat dibandingkan dengan objek lain (menggunakan metode `eq_()` atau `_cmp_()`). Objek hashable yang memiliki nilai hash yang sama harus memiliki hasil perbandingan yang sama.

Hashability memungkinkan objek digunakan sebagai kunci dalam struktur data seperti dictionary, karena struktur data ini menggunakan nilai hash secara internal.

Objek bawaan python biasanya hashable, sementara container dapat diubah (seperti list atau dictionary) tidak hashable. Objek yang merupakan instance dari kelas yang didefinisikan pengguna secara default hashable. Mereka semua membandingkan tidak sama, dan nilai hash mereka adalah id ().

Sintaks penulisan Tuple:

```
1  t1 = (1,2,3)
2  print(t1)
3  #output : (1,2,3)
```

Tuple dapat ditulis tanpa tanda kurung, tetapi lebih baik menggunakan tanda kurung untuk kejelasan

```
1 t1 = (1,)
2 print(t1)
3 #output (1,)
```

Tuple dengan satu elemen memerlukan koma di akhir. Jika tidak menambahkan koma maka akan dianggap sebagai tipe data string.

Membandingkan Tuple

Operator perbandingan (compare, seperti <,>==,dll.) dapat bekerja pada Tuple dan model sekuensial lainnya (list, dictionary, set). Proses perbandingan dimulai dengan membandingkan elemen pertama dari masing-masing sekuensial. Jika terdapat kesamaan, maka akan dilanjutkan ke elemen berikutnya, dan seterusnya, hingga menemukan perbedaan.

```
>>> (0, 1, 2) < (0, 3, 4)
True
>>> (0, 1, 2000000) < (0, 3, 4)
True
```

Fungsi sort di python juga bekerja dengan cara yang sama. Pertama-tama, akan dilakukan pengurutan berdasarkan elemen pertama. Jika ada elemen yang sama, maka akan dilanjutkan ke elemen kedua, dan seterusnya. Fitur ini disebut dengan DSU (Decorate, sort, undecorate):

1. **Decorate** : membangun daftar tuple dengan satu atau lebih kunci pengurutan sebelum elemen dari sekuensial.
2. **Sort** : mengurutkan daftar tuple menggunakan fungsi sort() bawaan python.
3. **Undecorate** : mengembalikan elemen yang telah diurutkan ke dalam sekuensial asli.

Contoh penggunaan DSU untuk mengurutkan kata dalam kalimat berdasarkan panjang kata:

```

1  kalimat = 'but soft what light in yonder window breaks'
2  dafkata = kalimat.split()
3  t = list()
4  for kata in dafkata:
5      t.append((len(kata), kata))
6  t.sort(reverse=True)
7  urutan = list()
8  for length, kata in t:
9      urutan.append(kata)
10 print(urutan)

```

Penugasan Tuple

Python memiliki fitur yang unik adalah kemampuannya untuk melakukan penugasan pada tuple di sisi kiri dari pernyataan penugasan. Hal ini memungkinkan kita untuk menetapkan lebih dari satu variabel secara berurutan.

Contoh penggunaan dua daftar elemen yang merupakan urutan:

```

1  m = [ 'have', 'fun' ]
2  x, y = m
3  print(x)
4  #output 'have'
5  print(y)
6  #output 'fun'

```

Tuple juga dapat digunakan untuk menukar nilai variabel dalam satu pernyataan :

```

1  m = [ 'have', 'fun' ]
2  x = m[0]
3  y = m[1]
4  print(x)
5  #output 'have'
6  print(y)
7  #output 'fun'

```

Tuple juga dapat digunakan untuk menukar nilai variabel dalam satu pernyataan:

```

1  a = 1
2  b = 2
3  a, b = b, a
4  print(a)
5  #output 2
6  print(b)
7  #output 1

```

Hal penting yang perlu diperhatikan yaitu jumlah variabel di sisi kiri dan sisi kanan harus sama:

```

1  try:
2  |   a, b = 1, 2, 3
3  except ValueError as e:
4  |   print(e)
5  #output: too many values to unpack (expected 2)

```

Dictionaries and Tuple

Dictionaries memiliki metode `items()` yang mengembalikan daftar tuple, di mana setiap tuple merupakan pasangan kunci-nilai.

Contoh :

```

1  d = {'a': 10, 'b': 1, 'c': 22}
2  t = list(d.items())
3  print(t)
4  #output : [('b', 1), ('a', 10), ('c', 22)]

```

Item yang dikembalikan nilainya tidak berurutan. Bagaimanapun juga, list dari tuple adalah list, dan tuple membandingkan jadi kita dapat melakukan pengurutan (`sort`) pada tuple. Melakukan konversi dictionary merupakan cara untuk menampilkan isi dictionary mengurutkan berdasarkan kunci.

```

1  d = {'a': 10, 'b': 1, 'c': 22}
2  t = list(d.items())
3  print(t)
4  #output : [('b', 1), ('a', 10), ('c', 22)]
5  t.sort()
6  print(t)
7  #output [('a', 10), ('b', 1), ('c', 22)]

```

Multipenugasan dengan dictionaries

Kombinasi antara items(), penugasan tuple, dan for loop dapat digunakan untuk mengakses kunci dan nilai dalam dictionary dalam satu loop. Contoh penggunaan nya adalah:

```
1 d = {'a': 10, 'b': 1, 'c': 22}
2 for key, val in list(d.items()):
3     print(val, key)
```

```
PS C:\pra_alpro\belajar> python -u "c:\pra_alpro\belajar\bd.py"
10 a
1 b
22 c
```

Kita dapat melakukan pencetakan isi dictionary yang diurutkan berdasarkan nilai yang disimpan di setiap pasangan key-value.

Kata yang sering muncul

Pada bagian ini kita akan mencoba menampilkan kata yang sering muncul dalam teks dari "Romeo and Juliet Act 2, scene 2". Silahkan unduh file Romeo-full.txt dari bagian materi. Kita akan membuat program menggunakan list dari tuple untuk menampilkan 10 kata yang sering muncul.

```
1 import string
2 fhand = open('romeo-full.txt')
3 counts = dict()
4 for line in fhand:
5     line = line.translate(str.maketrans('', '', string.punctuation))
6     line = line.lower()
7     words = line.split()
8     for word in words:
9         if word not in counts:
10             counts[word] = 1
11         else:
12             counts[word] += 1
13
14 #urutkan dictionary by value
15 lst = list()
16 for key, val in list(counts.items()):
17     lst.append((val, key))
18 lst.sort(reverse=True)
19 for key, val in lst[:10]:
20     print(key, val)
```

Penjelasan :

1. Membaca file : program membuka file Romeo-full.txt dan membaca setiap baris.
2. Pembersihan teks : setiap baris dibersihkan dari tanda baca menggunakan `str.maketrans` dan `translate`, kemudian diubah ke huruf kecil.
3. Penghitungan kata : setiap kata dalam baris dihitung menggunakan `dictionary counts`.
4. Mengurutkan kata : kata-kata diurutkan berdasarkan frekuensi kemunculan menggunakan list dari tuple.
5. Mencetak hasil : program mencetak 10 kata yang paling sering muncul.

Contoh Output :

```
61 i
42 and
40 romeo
34 to
34 the
32 thou
32 juliet
30 that
29 my
24 thee
```

Tuple sebagai kunci dictionaries

Tuple bersifat hashable, sehingga dapat digunakan sebagai kunci dalam dictionary. List tidak hashable, sehingga tidak dapat digunakan sebagai kunci. Ini sangat berguna ketika anda ingin membuat kunci komposit.

Contoh program :

```
1 directory = dict()
2 directory['Nendya', 'Dida '] = '088112266'
3 for last, first in directory:
4     print(first, last, directory[last, first])
```

Sumber : Materi 13 Tipe data Tuples (eclass)

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link GitHub : https://github.com/chei-cell/71251193_cheila-.git

SOAL 1

Source Code :

```
jmlh = int(input("Masukkan jumlah anggota list: "))

data = []

for c in range(jmlh):

    angka = int(input(f"Masukkan angka ke-{c+1}: "))

    data.append(angka)

c = all(x == data[0] for x in data)

print(c)
```

Langkah-langkah :

1. Langkah awal kita meminta pengguna untuk memasukkan jumlah data yang ingin dimasukkan ke list.
2. Membuat list kosong dengan nama variabel adalah **data**.
3. Program melakukan perulangan sebanyak jumlah data yang akan dimasukkan.
4. User akan memasukkan angka satu per satu, c+1 digunakan agar nomor urut dimulai dari 1 bukan 0.
5. Angka yang dimasukkan disimpan ke dalam list data

6. Program melakukan cek apakah semua anggota list memiliki nilai yang sama dengan elemen pertama.
7. Program akan menampilkan hasil pengecekan, **True atau False**.

Output :

```
PS C:\pra alpro\minggu13> python -u "c:\pra alpro\minggu13\soal01.py"
Masukkan jumlah anggota list: 4
Masukkan angka ke-1: 80
Masukkan angka ke-2: 80
Masukkan angka ke-3: 90
Masukkan angka ke-4: 90
False
```

SOAL 2

Source Code:

```
nama = input("Masukkan nama : ")
nim = input("Masukkan NIM : ")
alamat = input("Masukkan alamat : ")
data = (nama, nim, alamat)

print("\nDATA")

print("NIM      :", data[1])
print("NAMA     :", data[0])
print("ALAMAT:", data[2])

nim = tuple(data[1])
print("\nNIM:", nim)
```

```
nama_depan = tuple(data[0].split()[0])  
  
print("\nNAMA DEPAN:", nama_depan)  
  
nama_balik = tuple(reversed(data[0].split()))  
  
print("\nNAMA TERBALIK:", nama_balik)
```

Langkah-langkah :

1. Langkah pertama adalah meminta user untuk input nama, nim, dan alamat.
2. Nama, nim, dan alamat yang diinput oleh user disimpan ke dalam tuple data.
3. Program akan menampilkan tulisan data, digunakan untuk menampilkan judul agar output lebih rapi.
4. Program akan menampilkan nim, nama, dan alamat dari tuple.
5. Nim diubah menjadi tuple per karakter.
6. Program akan mengambil nama depan lalu mengubahnya menjadi tuple huruf. Split digunakan untuk memisahkan kata berdasarkan spasi, [0] untuk mengambil kata pertama, dan tuple() untuk mengubahnya menjadi tuple.
7. Program akan menampilkan tuple nama depan
8. Langkah terakhir, program membalik urutan nama dan menampilkan hasil

Output :

```
DATA
NIM   : 71251193
NAMA  : cheila zefanya wibowo
ALAMAT: jl klitren

NIM: ('7', '1', '2', '5', '1', '1', '9', '3')

NAMA DEPAN: ('c', 'h', 'e', 'i', 'l', 'a')

NAMA TERBALIK: ('wibowo', 'zefanya', 'cheila')
```

SOAL 3

```
file = input("Enter a file name: ")

handle = open(file)

jam_count = {}

for baris in handle:

    if baris.startswith("From "):

        kata = baris.split()

        waktu = kata[5]

        jam = waktu.split(":")[0]

        jam_count[jam] = jam_count.get(jam, 0) + 1

for jam, jumlah in sorted(jam_count.items()):

    print(jam, jumlah)
```

Langkah – langkah :

1. Langkah pertama, meminta pengguna untuk memasukkan nama file.
2. Lalu kita membuka file tersebut, dengan menggunakan **open()**
3. Kita membuat dictionary kosong, dengan variabel **jam_count**. Dictionary kosong ini untuk menyimpan jumlah email berdasarkan jam.

4. Program membaca file satu per satu setiap baris
5. Program memilih hanya baris yang diawali dengan kata "From"
6. Memisahkan baris menjadi beberapa bagian (list).
7. Mengambil data waktu dari indeks yang ke-5
8. Lalu, memisahkan waktu dengan berdasarkan tanda :. Hanya mengambil bagian jam saja
9. Menghitung berapa kali jam muncul. Get(jam, 0) jika belum ada mulai dari nol dan jika sudah ada ambil jumlah sebelumnya dan ditambah satu.
10. Mengurutkan isi dictionary berdasarkan jam
11. Menampilkan hasil jam dan jumlah email pada jam tersebut.

Output :

```
PS C:\pra alpro\minggu13> python -u "c:\pra alpro\minggu13\soal03.py"
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```